

Machine Learning

Model Building — Complete Study Notes

Simple explanations + Interview Q&A for every topic

Topics

1. Types of ML — Supervised vs Unsupervised
2. Supervised: Regression vs Classification
3. Linear Models — Linear & Logistic Regression, Gradient Descent
4. Nonlinear Models — Decision Tree, Random Forest, XGBoost
5. Model Evaluation — Regression & Classification Metrics
6. Hyperparameter Tuning — GridSearchCV & RandomizedSearchCV
7. Unsupervised — K-Means, DBSCAN, PCA

1. Types of Machine Learning

What is Machine Learning?

ML = teaching computers to learn from data instead of writing explicit rules. Just like you learn to recognize cats by seeing many cats, ML models learn patterns from examples (data) and make predictions on new unseen data.

The Big Picture

Machine Learning is broadly divided into three types:

	Data has labels?	Goal
Supervised	✅ Yes — data has correct answers	Predict outcomes
Unsupervised	❌ No labels	Find hidden patterns
Reinforcement	🏆 Rewards & penalties	Learn best action

Supervised Learning

Analogy
Like studying with an answer key. You see questions (features) + correct answers (labels).
The model learns the mapping: INPUT → OUTPUT from thousands of examples.

- Training data has BOTH inputs (X) and correct outputs (y)
- Model learns to predict y from X
- Two main tasks: Regression (predict a number) & Classification (predict a category)
- Examples: predicting house prices, spam detection, disease diagnosis

Unsupervised Learning

Analogy
Like exploring a new city without a map. You find your own patterns!
No labels — the model discovers structure hidden in raw data.

- Only inputs (X) — no correct answer labels
- Goal: discover clusters, associations, or reduced representations
- Examples: customer segmentation, anomaly detection, data compression

Interview Questions
Q1. What is the difference between supervised and unsupervised learning?
Q2. Can you give a real-world example of each type of ML?
Q3. What is the difference between ML and traditional programming?
Q4. When would you choose unsupervised over supervised learning?

2. Supervised: Regression vs Classification

The Core Difference — One Simple Question
REGRESSION → predicts a NUMBER (How much? How many? How far?)
CLASSIFICATION → predicts a CATEGORY (Which one? Is it A or B?)

Regression

- Output is a continuous number
- Examples: house price prediction, stock price, temperature forecast, age estimation
- Evaluation: MSE, MAE, RMSE, R^2 score

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)           # train
```

```
predictions = model.predict(X_test)    # predict continuous numbers
```

Classification

- Output is a discrete category/class
- Binary: 2 classes (spam/not spam, sick/healthy)
- Multiclass: 3+ classes (cat/dog/bird, digit 0-9)
- Evaluation: Accuracy, Precision, Recall, F1, ROC-AUC

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)     # predicts class labels: 0 or 1
probabilities = model.predict_proba(X_test)  # probability of each class
```

	Regression
Output type	Continuous number (e.g. 245,000)
Question answered	How much / How many?
Example	Predict house price: ₹45,00,000
Loss function	MSE, MAE, RMSE
	Classification
Output type	Discrete label (e.g. 'Spam')
Question answered	Which category / Is it X?
Example	Email → Spam or Not Spam
Loss function	Cross-entropy, Log loss

Interview Questions

Q1. What is the difference between regression and classification? Give examples.
Q2. Can a classification problem become a regression problem? How?
Q3. What is multi-label classification vs multi-class classification?
Q4. What loss functions are used for regression vs classification?

3. Linear Models

Linear Regression

Analogy — Best Fit Line

Imagine you're plotting house sizes vs prices on a graph.
Linear Regression draws the BEST straight line through those points.
Then for any new house size, you read off the price from that line!

The model learns: y = mx + b (or with multiple features: $y = w_1x_1 + w_2x_2 + ... + b$)

- y = prediction (house price)
- x = features (size, rooms, location)
- w (weights) = how much each feature matters
- b (bias) = where the line crosses the y-axis

Linear Regression Formula

$$y = w_1x_1 + w_2x_2 + ... + w_nx_n + b$$

y = output | x = features | w = weights | b = bias (intercept)

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Simple example: predict salary from years of experience
X = np.array([[1],[2],[3],[4],[5],[6],[7],[8],[9],[10]])  # years exp
y = np.array([35,40,50,55,65,72,80,90,95,100])           # salary (k)

model = LinearRegression()
model.fit(X, y)

print(f'Slope (weight): {model.coef_[0]:.2f}')           # ~7.2
print(f'Intercept (bias): {model.intercept_:.2f}')       # ~27.5
print(f'Predicted salary for 11 years: {model.predict([[11]])[0]:.1f}k')
```

Logistic Regression

Analogy — Sigmoid Switch

Despite the name, Logistic Regression is a CLASSIFICATION algorithm!
It uses a special S-shaped curve (sigmoid) to squish any number into 0→1.
If output > 0.5 → predict class 1 (e.g. spam). If < 0.5 → predict class 0.

Sigmoid Function

$$\sigma(z) = 1 / (1 + e^{-z}) \qquad \text{where } z = w \cdot x + b$$

Maps any real number to a probability between 0 and 1

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred):.3f}')  # ~0.96
```

	Linear Regression
Output	Continuous number
Algorithm	Minimise MSE
Use when	Predicting a quantity
	Logistic Regression
Output	Probability → class label
Algorithm	Maximise log-likelihood
Use when	Classifying into categories

Gradient Descent

Analogy — Finding the Valley

Imagine you're blindfolded on a hilly landscape and want to reach the lowest valley.
You feel the slope under your feet and take a small step downhill each time.
Gradient Descent does exactly this — takes small steps to minimize the error (loss)!

- Loss function measures how wrong the model is (e.g. MSE)
- Gradient = slope/direction of steepest increase in loss
- We move in the OPPOSITE direction of the gradient to reduce loss
- Learning rate (α) = how big each step is

Weight Update Rule

$$w = w - \alpha \times (\partial Loss / \partial w)$$

α = learning rate (step size) | ∂Loss/∂w = gradient of loss with respect to weight

	How it works	Best for
Batch GD	Uses entire dataset per step — very accurate but slow	Small datasets
Stochastic GD (SGD)	Uses 1 sample per step — fast but noisy	Online learning
Mini-batch GD	Uses small batch (32-256) — best of both worlds	Most common (Deep Learning)

```
# Gradient Descent from scratch (Linear Regression)
import numpy as np

def gradient_descent(X, y, lr=0.01, epochs=1000):
    m, n = X.shape
    w = np.zeros(n)   # weights
    b = 0             # bias
```

```
    for epoch in range(epochs):
        y_pred = X @ w + b                  # predictions
        error  = y_pred - y                 # error

        dw = (1/m) * X.T @ error            # gradient for weights
        db = (1/m) * np.sum(error)          # gradient for bias

        w -= lr * dw                        # update weights
        b -= lr * db                        # update bias

        if epoch % 100 == 0:
            loss = np.mean(error**2)
            print(f'Epoch {epoch:4d} | MSE Loss: {loss:.4f}')
    return w, b
```

⚠ Common Issues with Gradient Descent

Too large learning rate → overshoots the minimum (loss oscillates or explodes!)
Too small learning rate → takes forever to converge
Local minima → gets stuck in a valley that isn't the lowest
Solution: Use adaptive optimizers like Adam, RMSprop, Adagrad

Interview Questions

Q1. What is the difference between Linear and Logistic Regression?
Q2. Why is logistic regression called 'regression' if it's used for classification?
Q3. What is gradient descent? Explain with an analogy.
Q4. What is the learning rate and what happens if it's too high or too low?
Q5. What is the difference between Batch, Stochastic, and Mini-batch Gradient Descent?
Q6. What is the cost function for Linear Regression? For Logistic Regression?
Q7. What are the assumptions of Linear Regression?

4. Nonlinear Models (Tree-Based)

Why Tree-Based Models?

Linear models assume a straight-line relationship between features and output.
Real-world data is rarely linear! Tree models can capture complex
non-linear patterns and handle mixed data types without much preprocessing.

Decision Tree

Analogy — 20 Questions Game

You play 20 Questions to guess an animal. 'Is it a mammal?' → Yes.
'Does it live in water?' → No. 'Does it have stripes?' → Yes → TIGER!
A decision tree asks yes/no questions about features to reach a prediction.

- Root Node = first / most important split (question)
- Internal Nodes = decision points (more questions)

- Leaf Nodes = final prediction (answer)
- Split criterion: Gini Impurity (classification), MSE (regression)

Gini Impurity

$$Gini = 1 - \Sigma(p_i^2) \quad \text{where } p_i = \text{proportion of class } i$$

Measures how 'mixed' a node is. Gini=0 means perfectly pure (all one class)

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.datasets import load_iris

X, y = load_iris(data_only=False).data, load_iris().target

tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)

# Print the tree structure
print(export_text(tree, feature_names=load_iris().feature_names))

# Key hyperparameters
# max_depth       → limits tree depth (avoids overfitting)
# min_samples_split → min samples needed to split a node
# min_samples_leaf  → min samples in a leaf node
# criterion       → 'gini' or 'entropy' or 'mse'
```

✅ Pros	❌ Cons
Easy to visualise and interpret	Tends to overfit (grows too deep)
Handles numerical + categorical data	Unstable — small data change = different tree
No feature scaling needed	Biased toward high-cardinality features
Works for regression AND classification	Poor generalisation with deep trees

🌲 Random Forest

🌲🌲🌲 Analogy — Wisdom of the Crowd

One person's opinion can be wrong. But if you ask 500 people and take the majority vote, you'll likely get a better answer!
Random Forest builds MANY decision trees (an ensemble) and combines their predictions. Each tree is trained on a random subset of data AND features — reducing overfitting.

- Builds N decision trees (e.g. 100-500 trees)
- Each tree uses a random bootstrap sample of training data (bagging)
- Each split considers a random subset of features
- Final prediction: majority vote (classification) or average (regression)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
rf = RandomForestClassifier(
    n_estimators=200,     # number of trees
    max_depth=10,         # limit depth of each tree
    max_features='sqrt',  # features per split = sqrt(total features)
    random_state=42
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

# Feature importance — which features matter most?
importances = rf.feature_importances_
print(sorted(zip(feature_names, importances), key=lambda x: -x[1]))
```

⚡ XGBoost (Extreme Gradient Boosting)

> **🏆 Analogy — Learning from Mistakes**
> Imagine a student who gets exam answers wrong.
> After each attempt, they study ONLY the questions they got wrong and improve.
> XGBoost builds trees SEQUENTIALLY — each new tree focuses on fixing the errors of all previous trees. It's one of the most powerful ML algorithms!

- Boosting = build trees one after another, each correcting previous errors
- Uses gradient descent to minimise loss function at each step
- Regularisation (L1, L2) prevents overfitting — unlike vanilla Gradient Boosting
- Extremely fast due to parallelisation and cache-aware algorithms

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

xgb = XGBClassifier(
    n_estimators=300,      # number of boosting rounds
    learning_rate=0.05,    # shrinkage — smaller = more conservative
    max_depth=6,           # depth of each tree
    subsample=0.8,         # fraction of data per tree
    colsample_bytree=0.8,  # fraction of features per tree
    reg_alpha=0.1,         # L1 regularisation
    reg_lambda=1.0,        # L2 regularisation
    eval_metric='logloss',
    random_state=42
)

xgb.fit(X_train, y_train,
        eval_set=[(X_test, y_test)],
        early_stopping_rounds=20,  # stop if no improvement
        verbose=False)

print(f'Accuracy: {accuracy_score(y_test, xgb.predict(X_test)):.4f}')
```

	Type	Key Strength
Decision Tree	Single tree	Interpretable, easy to visualise
Random Forest	Bagging (parallel)	Reduces variance, robust to outliers

XGBoost	Boosting (sequential)	State-of-the-art accuracy, handles missing values

Interview Questions

Q1. What is a Decision Tree and how does it decide where to split?
Q2. What is Gini Impurity? What is Information Gain / Entropy?
Q3. How does a Random Forest differ from a single Decision Tree?
Q4. What is bagging vs boosting? How do they reduce error?
Q5. What is XGBoost and why is it better than Random Forest?
Q6. What is the difference between variance and bias in tree models?
Q7. How does XGBoost handle missing values?
Q8. What are the key hyperparameters of Random Forest and XGBoost?

5. Model Evaluation

Regression Metrics

Mean Absolute Error (MAE)

$$MAE = (1/n) \times \Sigma |y_i - \hat{y}_i|$$

Average of absolute differences. Easy to interpret — in same units as target. Less sensitive to outliers.

Mean Squared Error (MSE)

$$MSE = (1/n) \times \Sigma (y_i - \hat{y}_i)^2$$

Squares the errors — penalises large mistakes HEAVILY. Always ≥ 0. Sensitive to outliers.

Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE} = \sqrt{[(1/n) \times \Sigma (y_i - \hat{y}_i)^2]}$$

Same units as the target variable. More interpretable than MSE. Popular for reporting.

Mean Absolute Percentage Error (MAPE)

$$MAPE = (100/n) \times \Sigma |(y_i - \hat{y}_i) / y_i| \quad \%$$

Error as a percentage — great for comparing across scales. Problem: blows up when $y_i \approx 0$.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_true = [100, 200, 300, 400, 500]
y_pred = [110, 195, 290, 420, 480]

mae  = mean_absolute_error(y_true, y_pred)
mse  = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
mape = np.mean(np.abs((np.array(y_true)-np.array(y_pred))/np.array(y_true)))*100
r2   = r2_score(y_true, y_pred)
```

```
print(f'MAE:  {mae:.2f}')    # 14.0
print(f'MSE:  {mse:.2f}')    # 270.0
print(f'RMSE: {rmse:.2f}')   # 16.43
print(f'MAPE: {mape:.2f}%')  # 5.95%
print(f'R²:   {r2:.4f}')     # 0.9946  (1.0 = perfect)
```

Classification Metrics

Confusion Matrix — The Foundation of All Classification Metrics

A 2×2 table showing predicted vs actual outcomes for binary classification:

TP (True Positive) = predicted YES, actual YES ✅
TN (True Negative) = predicted NO, actual NO ✅
FP (False Positive) = predicted YES, actual NO ❌ (Type I Error)
FN (False Negative) = predicted NO, actual YES ❌ (Type II Error)

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
print(cm)
# [[TN  FP]
#  [FN  TP]]

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title('Confusion Matrix')
plt.show()
```

Accuracy

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Overall correctness. Misleading when classes are imbalanced! (e.g. 99% accuracy if always predict majority)

Precision

Precision = TP / (TP + FP)

Of all positives predicted, how many were actually positive? → 'Don't cry wolf!'

Recall (Sensitivity)

Recall = TP / (TP + FN)

Of all actual positives, how many did we catch? → 'Don't miss real cases!' (critical in medical diagnosis)

F1 Score

F1 = 2 × (Precision × Recall) / (Precision + Recall)

Harmonic mean of Precision and Recall. Best metric for imbalanced datasets.

Precision vs Recall — When to use which?

HIGH PRECISION matters when: False Positives are costly
→ Spam filter: don't mark real emails as spam!
HIGH RECALL matters when: False Negatives are costly
→ Cancer detection: don't miss a real cancer patient!
F1 Score: use when you need to balance both (imbalanced data)

ROC Curve & AUC

- ROC = Receiver Operating Characteristic curve
- Plots True Positive Rate (Recall) vs False Positive Rate at different thresholds
- AUC (Area Under Curve): 1.0 = perfect, 0.5 = random guessing, < 0.5 = worse than random
- Use AUC when you need a threshold-independent performance measure

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve)
import matplotlib.pyplot as plt

print(f'Accuracy:  {accuracy_score(y_test, y_pred):.4f}')
print(f'Precision: {precision_score(y_test, y_pred):.4f}')
print(f'Recall:    {recall_score(y_test, y_pred):.4f}')
print(f'F1 Score:  {f1_score(y_test, y_pred):.4f}')
print(f'ROC-AUC:   {roc_auc_score(y_test, y_prob[:,1]):.4f}')

# Plot ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_prob[:,1])
plt.plot(fpr, tpr, label=f'AUC = {roc_auc_score(y_test, y_prob[:,1]):.2f}')
plt.plot([0,1],[0,1],'--', label='Random')
plt.xlabel('False Positive Rate');  plt.ylabel('True Positive Rate')
plt.title('ROC Curve');  plt.legend();  plt.show()
```

Interview Questions

Q1. What is the difference between MAE, MSE, and RMSE? When would you prefer each?
Q2. What is MAPE and when does it fail?
Q3. What is a Confusion Matrix? Explain TP, FP, TN, FN.
Q4. What is the difference between Precision and Recall? Give real-world examples.
Q5. When would you optimise for Precision over Recall and vice versa?
Q6. What is the F1 Score and why is it better than accuracy for imbalanced datasets?
Q7. What is the ROC curve and what does AUC represent?
Q8. If your dataset has 95% of class 0 and 5% of class 1, which metric should you use?

6. Hyperparameter Tuning

Parameters vs Hyperparameters

PARAMETERS → learned automatically during training (e.g. weights, biases)
HYPERPARAMETERS → set by YOU before training (e.g. learning_rate, max_depth, n_estimators)
Tuning hyperparameters = finding the best settings for your model.

GridSearchCV — Exhaustive Search

Analogy

Imagine tasting every single combination of flavours at an ice cream shop.
GridSearch tries EVERY combination of hyperparameters you give it.
Guaranteed to find the best combo — but can be VERY slow!

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

# Define the hyperparameter grid
param_grid = {
    'n_estimators': [100, 200, 300],     # 3 options
    'max_depth':    [5, 10, 15, None],   # 4 options
    'max_features': ['sqrt', 'log2'],    # 2 options
    # Total combinations = 3 × 4 × 2 = 24
}

rf = RandomForestClassifier(random_state=42)

grid_search = GridSearchCV(
    estimator  = rf,
    param_grid = param_grid,
    cv         = 5,          # 5-fold cross-validation
    scoring    = 'f1',       # optimise for F1 score
    n_jobs     = -1,         # use all CPU cores
    verbose    = 1
)

grid_search.fit(X_train, y_train)

print('Best parameters:', grid_search.best_params_)
print('Best F1 Score:  ', grid_search.best_score_)

best_model = grid_search.best_estimator_
```

RandomizedSearchCV — Smart Random Search

Analogy

Instead of tasting every combination, you randomly sample 20 combos.
Surprisingly, this works nearly as well as Grid Search but much FASTER!
RandomizedSearchCV samples a fixed number of parameter combinations.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

# Can use distributions, not just fixed values!
param_dist = {
    'n_estimators':     randint(50, 500),     # any int from 50-500
    'max_depth':        randint(3, 20),        # any int from 3-20
    'max_features':     ['sqrt', 'log2'],
    'min_samples_leaf': randint(1, 10),
    'learning_rate':    uniform(0.01, 0.3),   # XGBoost: float 0.01-0.31
}
```

```
random_search = RandomizedSearchCV(
    estimator           = rf,
    param_distributions= param_dist,
    n_iter              = 50,    # try 50 random combos (not all!)
    cv                  = 5,
    scoring             = 'f1',
    n_jobs              = -1,
    random_state        = 42,
    verbose             = 1
)

random_search.fit(X_train, y_train)
print('Best params:', random_search.best_params_)
```

	GridSearchCV
Approach	Try every combination exhaustively
Speed	Slow — O(n^k) combinations
Best for	Small parameter grids
Guaranteed best?	Yes (within the grid)
	RandomizedSearchCV
Approach	Randomly sample n_iter combinations
Speed	Fast — fixed number of trials
Best for	Large parameter spaces
Guaranteed best?	No — probabilistic

Cross-Validation (CV) explained

CV = evaluating the model multiple times on different train/test splits.
cv=5 means: split data into 5 parts. Train on 4, test on 1. Repeat 5 times.
Average the 5 scores → more reliable than a single train/test split!

Interview Questions

Q1. What is the difference between a parameter and a hyperparameter?
Q2. What is cross-validation and why is it important?
Q3. What is GridSearchCV and how does it work?
Q4. What is RandomizedSearchCV and how does it differ from GridSearchCV?
Q5. When would you prefer RandomizedSearchCV over GridSearchCV?
Q6. What is overfitting and how does hyperparameter tuning help prevent it?
Q7. What other hyperparameter optimisation techniques exist beyond Grid/Random search?

7. Unsupervised Learning

K-Means Clustering

Analogy — Sorting Cities into Regions

Imagine dropping 5 pins randomly on a map. Every city 'belongs' to its nearest pin.
Move each pin to the centre of its group. Repeat until pins stop moving.
K-Means assigns data points to K clusters and iterates until stable!

How K-Means Works (Step by Step)

- Step 1: Choose K (number of clusters)
- Step 2: Randomly initialise K centroids (cluster centres)
- Step 3: Assign each point to the nearest centroid (using Euclidean distance)
- Step 4: Recompute centroids as the mean of all points in each cluster
- Step 5: Repeat Steps 3-4 until centroids stop moving (convergence)

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Always scale data before K-Means!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Finding optimal K using the Elbow Method
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)    # sum of squared distances to centroid

plt.plot(range(1,11), inertias, 'bo-')
plt.xlabel('Number of Clusters K')
plt.ylabel('Inertia (within-cluster sum of squares)')
plt.title('Elbow Method — choose K at the elbow!')
plt.show()

# Fit with chosen K
km = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = km.fit_predict(X_scaled)
print('Cluster labels:', labels)    # e.g. [0, 2, 1, 0, 2, ...]
```

✓ Pros	✗ Cons
Simple and fast	Must specify K in advance
Scales to large datasets	Sensitive to initial centroids
Easy to interpret	Assumes spherical clusters
Works well with compact clusters	Sensitive to outliers

DBSCAN

Analogy — Finding Neighbourhoods in a City

DBSCAN = Density-Based Spatial Clustering of Applications with Noise.

Think of crowded city blocks (clusters) vs empty outskirts (noise/outliers).
Areas with many nearby points = clusters. Isolated points = noise.
Best part: you DON'T need to specify K! It finds clusters automatically.

- eps = neighbourhood radius (how close points must be to be 'neighbours')
- min_samples = minimum points to form a 'dense' region (a cluster)
- Core Point: has ≥ min_samples within eps radius
- Border Point: within eps of a core point but has fewer than min_samples neighbours
- Noise Point (outlier): not core, not border — labelled as -1

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

dbscan = DBSCAN(
    eps=0.5,            # neighbourhood radius
    min_samples=5       # min points to be a core point
)
labels = dbscan.fit_predict(X_scaled)

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise    = list(labels).count(-1)

print(f'Clusters found: {n_clusters}')
print(f'Noise points:   {n_noise}')
print(f'Labels: {set(labels)}')   # e.g. {-1, 0, 1, 2}  (-1 = noise)
```

	K-Means	DBSCAN
K required?	✅ Yes	❌ No — auto-detects
Cluster shape	Spherical only	Any shape!
Handles outliers?	❌ No — assigns to cluster	✅ Yes — labels as noise
Sensitive to	Initial centroids, K choice	eps & min_samples values
Best for	Compact, similar-size clusters	Irregular shapes, noisy data

PCA — Principal Component Analysis

Analogy — Photographing a 3D Object

Imagine a 3D sculpture. A photo captures it in 2D but keeps the most important details.
PCA does the same for data: reduces many features to fewer dimensions
while retaining as much VARIANCE (information) as possible.
Used for: speeding up training, visualisation, removing noise, feature extraction.

- Principal Components = new axes that capture the most variance in data
- PC1 = direction of maximum variance. PC2 = second most variance (perpendicular to PC1)
- Explained Variance Ratio = how much of the total variance each PC captures
- Choose n_components to keep ~95% of variance

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

# Step 1: Always scale first!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 2: Fit PCA and see explained variance
pca = PCA()
pca.fit(X_scaled)

# Cumulative explained variance
cumvar = np.cumsum(pca.explained_variance_ratio_)
plt.plot(range(1, len(cumvar)+1), cumvar, 'bo-')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% threshold')
plt.xlabel('Number of Components');  plt.ylabel('Cumulative Variance')
plt.title('PCA — How many components to keep?')
plt.legend();  plt.show()

# Step 3: Apply PCA keeping 95% variance
pca_95 = PCA(n_components=0.95)   # auto-select components for 95% var
X_reduced = pca_95.fit_transform(X_scaled)
print(f'Original features: {X_scaled.shape[1]}')
print(f'Reduced features:  {X_reduced.shape[1]}')

# Step 4: 2D visualisation
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_scaled)
plt.scatter(X_2d[:,0], X_2d[:,1], c=labels, cmap='tab10')
plt.xlabel('PC1');  plt.ylabel('PC2')
plt.title('PCA — 2D Projection');  plt.show()
```

Interview Questions

Q1. What is the difference between K-Means and DBSCAN?
Q2. How do you choose the value of K in K-Means? Explain the Elbow Method.
Q3. What are the limitations of K-Means clustering?
Q4. How does DBSCAN identify outliers/noise?
Q5. What is PCA and why do we need it?
Q6. Why must you scale data before applying PCA or K-Means?
Q7. What is explained variance ratio in PCA?
Q8. What is the curse of dimensionality and how does PCA help?
Q9. Can PCA be used as a preprocessing step before a classification model? Why?

Master Cheat Sheet

Model Selection Guide

	Model Options	Start With
Regression	Linear Reg, RF, XGBoost	Linear Reg → XGBoost
Binary Classification	Logistic Reg, RF, XGBoost	Logistic Reg → XGBoost

Multiclass Classification	Softmax, RF, XGBoost	RF → XGBoost
Clustering	K-Means, DBSCAN	K-Means → DBSCAN if shapes vary
Dimensionality Reduction	PCA, t-SNE, UMAP	PCA first, t-SNE for viz

Metrics Quick Reference

	Formula (simplified)	Use when
MAE	avg|y - ŷ|	Outliers present, interpretable
RMSE	√avg(y-ŷ)²	Penalise large errors
MAPE	avg|y-ŷ|/|y| × 100%	Relative % error comparison
Accuracy	(TP+TN)/Total	Balanced classes only
Precision	TP/(TP+FP)	Minimise false alarms
Recall	TP/(TP+FN)	Minimise missed positives
F1	2×P×R/(P+R)	Imbalanced classes
AUC-ROC	Area under TPR/FPR curve	Threshold-independent eval

Key Sklearn Code Snippets

```
# ── TRAIN / TEST SPLIT ──────────────────────────────────────
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# ── SCALING ─────────────────────────────────────────────────
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)   # fit+transform on train
X_test  = scaler.transform(X_test)         # only transform on test!

# ── CROSS-VALIDATION ────────────────────────────────────────
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='f1')
print(f'CV F1: {scores.mean():.4f} ± {scores.std():.4f}')

# ── PIPELINE ────────────────────────────────────────────────
from sklearn.pipeline import Pipeline
pipe = Pipeline([('scaler', StandardScaler()), ('model', XGBClassifier())])
pipe.fit(X_train, y_train)
pipe.predict(X_test)
```

Top Interview Questions — Quick Answers

Bias-Variance Tradeoff?	High bias = underfitting (too simple). High variance = overfitting (too complex). Sweet spot = balanced model.
Overfitting vs Underfitting?	Overfit: great train score, poor test. Underfit: poor both. Fix: regularisation, more data, simpler model.

Why scale data?	Distance-based models (KMeans, PCA, SVM) are sensitive to feature magnitudes. Scale so no feature dominates.
What is regularisation?	L1 (Lasso) can zero out weights. L2 (Ridge) shrinks all weights. Both prevent overfitting.
Train vs Val vs Test split?	Train: model learns. Validation: tune hyperparameters. Test: final unbiased evaluation. Never tune on test!

🎓 You're now ML Interview Ready! 🚀

Practice on real datasets (Kaggle) and implement every algorithm from scratch at least once!